libmemctl

SYSPROG10 / Systems Programming / Technical report

Jashandeep Singh - solo participant - IMT2024022
13 November 2026

A C11 Linux allocator providing the four required my_* functions through private anonymous mappings. The current source combines reusable small-object regions, eight shards and capped dedicated-mapping reuse for requests above 64 KiB through 1 MiB. The report covers design, measured trade-offs and executed validation; no official score is claimed.

Measured improvement and cost

Five fresh processes per case on a shared i3-1315U laptop, GCC 15.2.0 and glibc 2.43. The comparison uses the pinned baseline without large-mapping reuse, with identical traces and flags. Small/mixed throughput below uses the longer five-million-call confirmation.

Throughput (M API calls/s)	Threads	Old	Current
Small	1	38.277	37.293
Small	8	60.753	59.870
Mixed	1	29.486	30.650
Mixed	8	57.639	51.240
Large	1	0.165	0.235
Large	8	0.0437	0.0740

Large at eight threads: throughput median rose 69.6%; malloc p99 fell from 1,784.819 to 1,105.217 microseconds (38.1%). **Costs:** eight-thread mixed-throughput median fell 11.1%, and held virtual memory in the full-touch large burst rose 34.7%. The corresponding resident-memory growth was nearly unchanged. One-thread large p99 rose 8.3%.

The mixed-throughput min-max ranges overlap, and clock, scheduling, thermal and desktop noise remain. These observations do not establish a universal speedup. See docs/report.md and results/benchmarks/ for the methodology, counts, seeds and ranges. The paired comparison was captured on 9 November 2026.

Design and correctness

Core invariants

Ownership: each region has one immutable shard/class owner; valid objects keep their backing region live. Cross-thread free follows that owner, including after producer exit.

Locking: per-bin locks protect lists, counts and cache slots. No operation holds two allocator locks; realloc allocates/copies before releasing the old object.

Space: 64 small classes cover requests through 64 KiB. Regions start compact, then grow after demand. Large reuse uses 64-KiB bands through 1 MiB; larger requests map directly.

Reclamation: all empty caches share a 2 MiB reservation bound including mapping overhead. Trim removes only unused mappings. Sparse live objects can still pin large partially occupied regions.

API: checked arithmetic, max alignment, explicit calloc zeroing and failure-preserving realloc. The implementation does not forward calls to the standard heap allocator.

Executed evidence

Environment / check	Result and scope
Ubuntu 26.04.1, GCC 13.4 / GCC 15.2	Native independent and six official tests; ASan/UBSan controls passed.
ASan control subprocesses	59/59: 25 valid and 34 detected invalid accesses, including cached large boundaries.
GCC 15 stress / TSan	Two million native mixed operations plus handoffs; ASan/UBSan and TSan soaks passed.
NHPC GCC 9.3.1 / glibc 2.17	Current source built on login host; allocated compute job passed independent checks, one million operations and 131,072 handoffs.
Linux 3.10 compatibility	Full-touch memory probe passed using a summed smaps RSS fallback; no cluster performance comparison.

Sanitizers do not prove correctness. LeakSanitizer was disabled for the documented host limitation. Address reuse can hide stale-pointer access; invalid/double frees are outside the API contract. Only the authored allocator mappings are covered by the independent reclamation accounting.

Evaluate and reproduce

One-command evaluation

From the repository root, run the isolated evaluator. It verifies source and artifact integrity, builds fresh objects, checks forbidden allocator references, runs independent correctness tests and compiles/executes the public-API example. Each run saves exact commands, compiler version, stdout/stderr, exit codes and the source hash in its own build directory.

```
python3 scripts/evaluate.py
python3 scripts/evaluate.py --sanitizers
```

Only the optional --official-dir argument needs the separately obtained official public tests. The default check is offline. Linux x86-64, a C11 compiler, Make, binutils, pthreads, coreutils and Python 3.9+ are required; report/plot/video tooling is optional.

Evidence map

Artifact	Location
Implementation / interface	src/allocator.c; include/libmemctl.h
Design / methodology	docs/architecture.md; docs/report.md
Throughput, latency and memory	results/benchmarks/
Correctness and stress	results/correctness/
Executed Linux environments	results/compatibility/
Library / test executables	prebuilt/ (source and binary checksums included)
Functional demonstration	demo/run_demo.sh; demo/terminal-replay.mp4

The results index links the selected paired comparisons, longer common-workload confirmation and current-source 1/2/4/8-thread scaling. Raw records preserve capture-time identifiers and paths. The supplied baseline has a pinned source hash and is compiled separately for comparisons. Current reproduction commands are in docs/reproduction.md.

Attribution and sources

AI assistance was used for implementation, testing, analysis and documentation. Permission is attributed to the participant's report of SPOC phone confirmation, not to a published AI-code clause. The implementation and independent tests were authored for the permitted event window. Official kit copies are not redistributed.

Source, evaluator and recorded results
Official selected statement, pinned d23ac1d

Allocator SHA-256:

```
ce2e34f557608c4c1cd4277a12dc85fa88cdcc343e6540d74924ddd31024d267
```

The video replays an actual preserved terminal run; it is not a desktop screen capture. No official normalization targets or hidden-test results are inferred. Benchmark limitations and the throughput/space costs are retained in the full report.